Selecting Reliable LLM Reasoning: Majority Vote vs. a Learned Verifier

DASE7506 · Advanced Machine Learning Methods · MP2
Zhao Jingchun · 3036708440

Abstract

A 0.6B model produced four candidate solutions to each of 6,020 maths problems; the task is to pick one per problem, scored on the chosen candidate's *final numeric answer*. I built a linear scorer over the supplied 1024-d solution embeddings, 17 text statistics and 9 numeric-verification features, trained with a listwise softmax loss, and added the design decision that mattered most: **pooling candidate scores by answer before choosing**. The frozen method reaches **83.30% on the official test bank** (389/467), against 70.70% for majority voting on the validation hard subset. All gains over voting come from overturning wrong majorities and breaking true ties; 81% of the remaining errors are solutions with correct arithmetic that dropped a condition or misread the question. Ablation shows the embedding carries most of the signal (−6.86 points when removed), length and formatting contribute nothing once it is present, and more elaborate alternatives (within-question features, a boosted second stage, ensembling, an MLP) all failed to beat the linear model on train cross-validation.

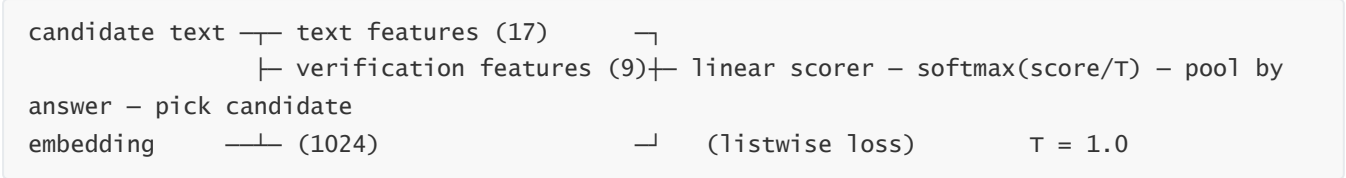
1. Task and constraints

Splits	4,755 train / 798 validation / 467 test questions, four candidates each, no overlap
Inputs	question text, four candidate solutions, fixed embeddings `[N, 4, 1024]` from Qwen3-0.6B
Metric	selection accuracy on the full test bank (%)
Inference budget	≤ 16 MiB of assets; ≤ 30 s for prediction **and** scoring on 4 CPU threads, including startup
Prohibited	generating new text, external data, additional pretrained weights, ID-based lookup; test references may not enter training, tuning or method selection

The test bank is entirely "hard": every test question has at least two distinct parsed answers, whereas 56% of validation questions are easy (all four candidates agree). Full-validation accuracy therefore overstates test performance, so I also report a **hard subset** (355 validation questions with ≥ 2 distinct answers); §3.5 shows this proxy is sound. **Credit is outcome-based**: a candidate can compute flawlessly and still be wrong, and §5.3 shows this is the dominant failure mode.

2. Method

2.1 Pipeline and evidence blocks



Block	Dim	What it captures
`emb`	1024	the supplied whole-solution embedding
`text`	17	whether the answer parses / is an integer, coverage of the question's numbers, length, line and equation counts, formatting flags, hedging words
`verify`	9	arithmetic re-checked exactly, plus answer–process consistency

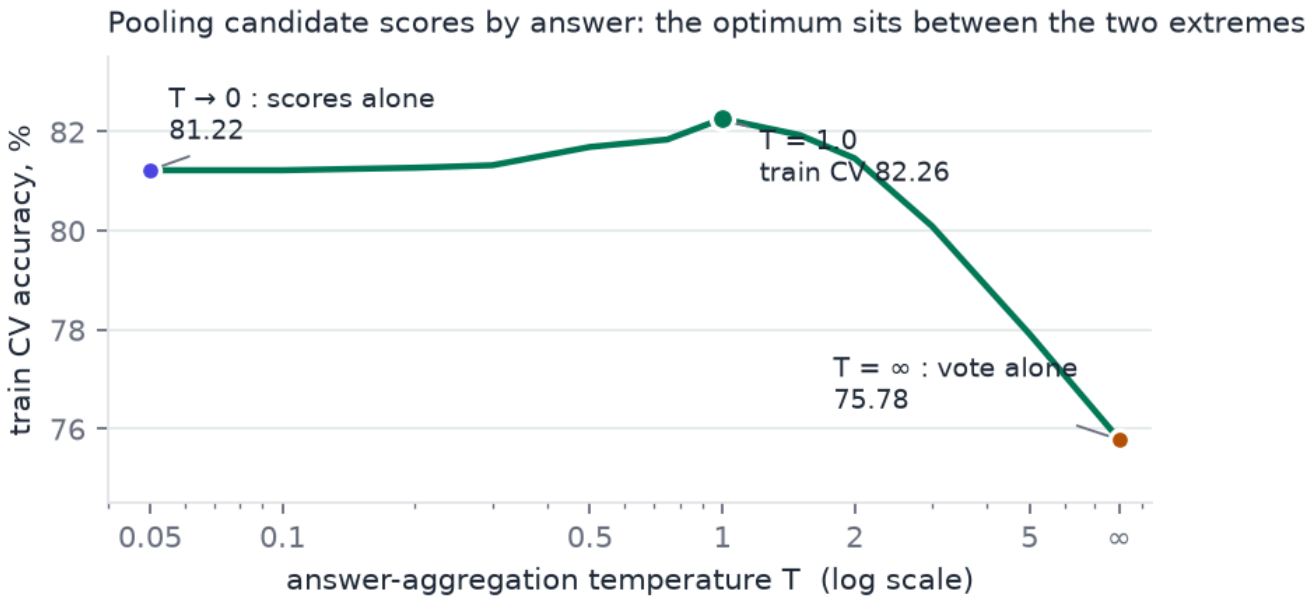
The verifier normalises LaTeX, units and markdown and evaluates an equation only when its left-hand side reduces to pure-number arithmetic, using an AST walk with `Fraction` (never `eval`); anything symbolic is skipped, not flagged. Five rounds of manual audit on 1,200 candidates cut the flagged set from 187 lines to 12, all genuine errors (e.g. `45 + 90 + 135 + 45 + 60 = 395`). Removed false positives were unit conversions, rearrangements, percentages, symbolic sides and thousands separators.

2.2 Training objective

The scorer minimises a **listwise softmax** loss over a question's four candidates, maximising the probability mass on correct candidates (there may be several). `C = 0.03`, chosen by 5-fold cross-validation on train with a fixed seed. It beats the pointwise per-candidate alternative (84.23% vs 82.54% on the validation hard subset) because the deployed decision is a within-question comparison, which the listwise loss optimises directly.

2.3 Pooling scores by answer — the decision that mattered

Each candidate receives weight `softmax(score / T)` within its question; candidates with the **same parsed answer** pool their weights, the heaviest answer wins, and its highest-scoring candidate is returned. `T` interpolates between the scorer alone (`T → 0`) and plain majority voting with score tie-breaks (`T = ∞`), and is chosen on train out-of-fold scores only.



The optimum lies in the middle (`T = 1.0`): dropping the scores (∞) costs 6.5 points and dropping the vote structure (→0) costs 1.0. The two signals are complementary, and pooling lets both act at once.

A structural constraint. For a linear scorer, subtracting the question mean from a feature never changes the within-question argmax, so cross-candidate information cannot enter through mean-centred features — only through non-linear statistics such as votes and ranks. Answer pooling is such a non-linear aggregation, which is why it worked where engineered within-question features did not (§3.2).

---

3. Experiments

All method and hyper-parameter choices were made on **train cross-validation** (5-fold, seed 0, 2,114 hard questions); validation served for confirmation (§3.5 discusses the resulting optimism).

3.1 Baselines and progressive improvements

Method	Validation, all	Validation, hard	Train CV, hard
Always pick candidate 0	78.20%	54.93%	—
Majority vote (longest solution breaks ties)	86.97%	70.70%	69.91%
Majority vote, dropping non-integer answers	88.60%	74.37%	—
Per-candidate scorer, text features only	82.96%	61.69%	59.74%
Per-candidate scorer, embedding only	89.22%	76.06%	78.43%
Per-candidate scorer, embedding + text	92.23%	82.54%	80.75%
Listwise scorer, embedding + text	92.98%	84.23%	80.70%
Listwise scorer, + within-question features	92.86%	83.94%	81.41%
Listwise scorer, + verification features	93.36%	85.07%	81.17%

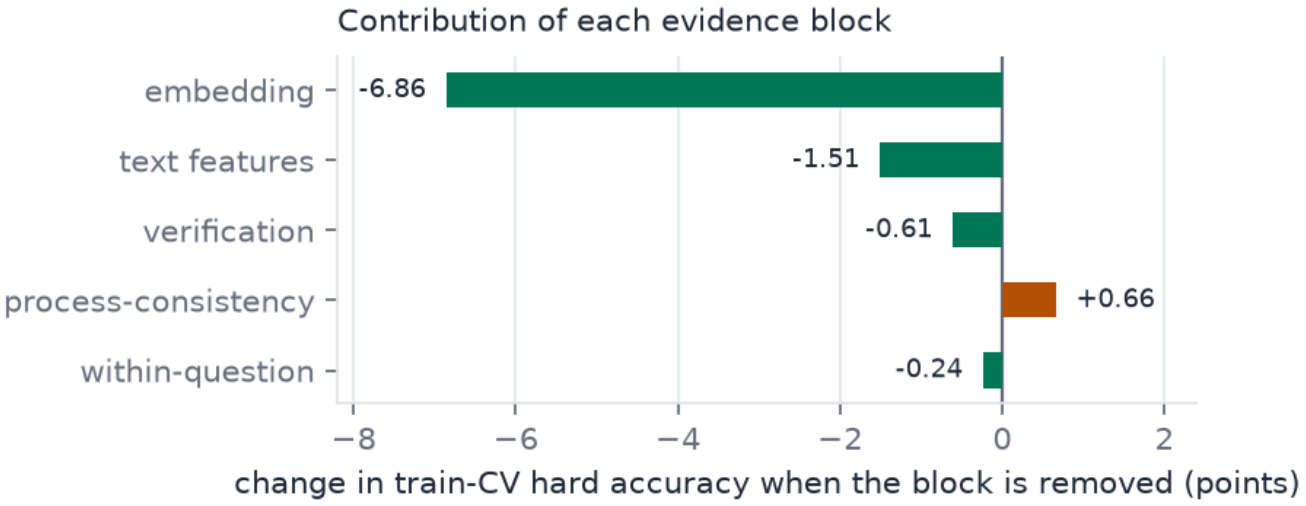
Method	Validation, all	Validation, hard	Train CV, hard
Final: listwise scorer + answer pooling	**93.86%**	**86.20%**	**82.26%**

Paired McNemar tests on the validation hard subset: final method vs. majority vote 65 wins / 12 losses ($p = 6 \times 10^{-10}$); per-candidate scorer vs. vote 64 / 22 ($p = 7 \times 10^{-6}$); text-only scorer vs. vote 48 / 80 ($p = 0.006$, worse). The gain over the best untrained baseline is unambiguous; between the intermediate rows, differences of 1–2 points on 355 questions are about one standard error and I do not claim them as significant.

3.2 Ablation: which evidence actually contributes

Each variant uses the identical procedure (listwise, `C = 0.03`, `T = 1.0`).

Configuration	Dim	Train CV hard	Validation hard
Full: embedding + text + verification	1050	**82.26**	**86.20**
without embedding	26	75.40	80.85
without text features	1033	80.75	82.82
without verification	1041	81.65	85.07
without `ans_integer` only	1048	80.75	84.79
embedding only	1024	79.56	82.25
text only / verification only	17 / 9	73.79 / 67.98	79.44 / 69.58
plus within-question features	1060	82.02	86.20
plus process-consistency features	1060	82.92	86.76
embedding + length / + formatting only	1026 / 1034	79.56 / 79.52	82.25 / 82.82
length features alone	2	67.69	68.17

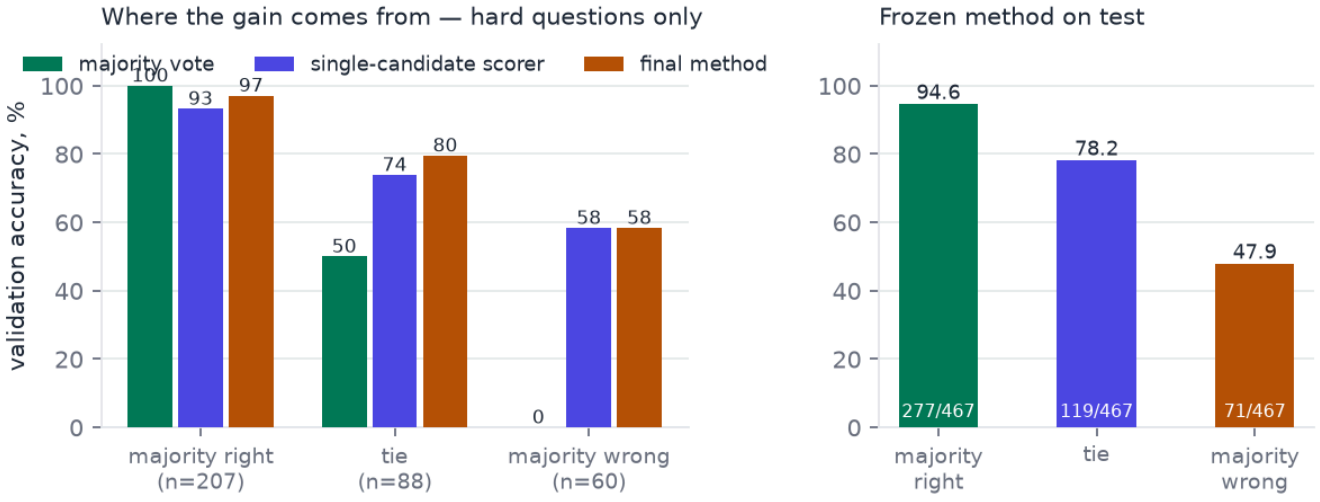


- **The embedding carries almost all of the signal**: removing it costs 6.9 points; no other block is worth more than 1.6.

- **Text features pay off only in combination**: alone they score 61.69% on the hard subset, *below* voting, yet add 1.5 points on top of the embedding, chiefly through whether the answer parses and is an integer.
- **The verifier is small but measurable** (−0.61), consistent with §4: the model's mistakes are rarely arithmetic.
- **Within-question features contribute nothing.** Given only this block, a linear model reproduces majority voting exactly (identical 86.97 / 70.70 on validation); under these features voting is already the optimal linear rule, and adding them to the full model changes nothing (82.02 vs 82.26).
- **Length and formatting contribute nothing measurable.** `embedding + length` reproduces `embedding only` to the decimal; the formatting variant differs by −0.04 on CV and +0.57 on validation, i.e. no consistent direction. With the embedding present the length coefficients are about +0.07, essentially zero, whereas a text-only model leans on `log_numbers` (+0.83) and `ans_integer` (+0.84). Length alone scores 68.17%, below voting. With the embedding available, the method does not rely on surface cues.

3.3 Where the gain comes from, and where it stops

Question type	Val n	Majority vote	Per-candidate scorer	Final	Test n	Final (test)
Majority right	207	100.0%	93.2%	97.1%	277	94.6%
True tie	88	50.0%	73.9%	79.5%	119	78.2%
Majority wrong	60	0.0%	58.3%	58.3%	71	47.9%

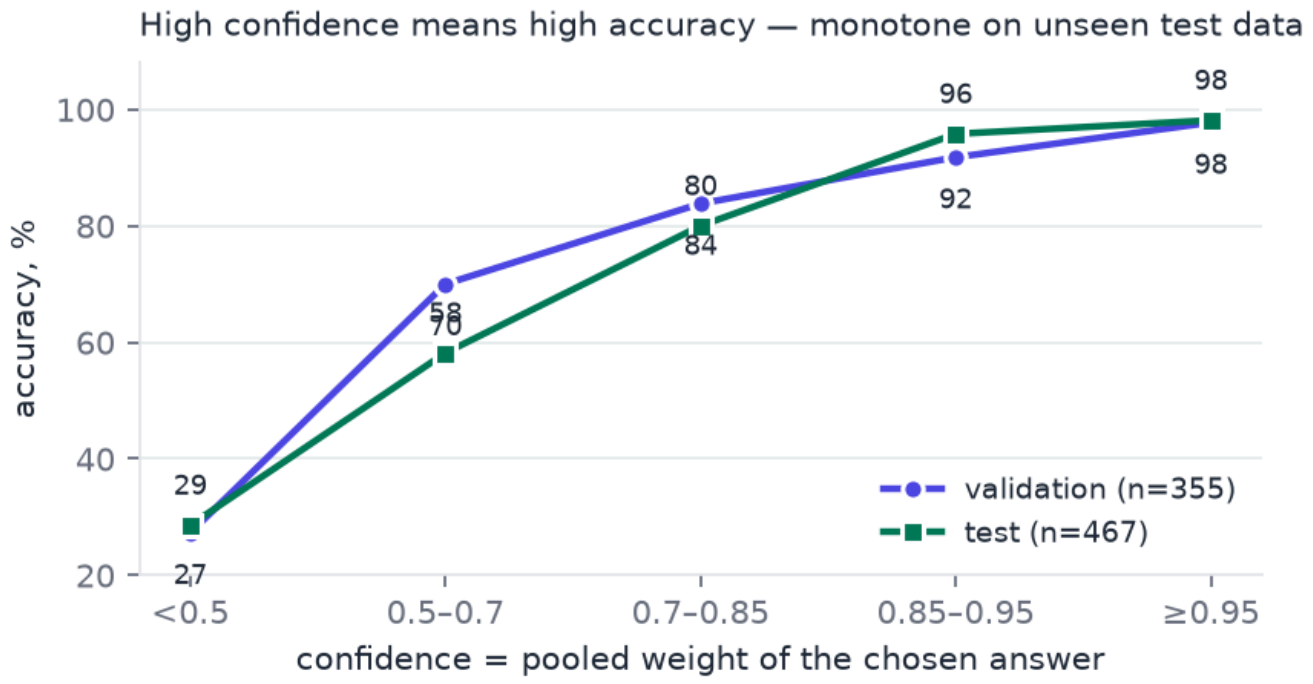


The learned scorer earns its keep exactly where voting cannot: it recovers about 35 of the 60 wrong-majority questions and about 26 of the 88 true ties, at the cost of overturning a few correct majorities. Pooling shifts this balance toward majorities that also have content support: majority-right accuracy rises from 94.2% to 97.1% while majority-wrong falls from 65.0% to 58.3%; because the first group is 3.5× larger, the trade is net positive. On test, **63 of the 78 errors (81%) fall in the tie and majority-wrong groups**, where the vote gives no help.

3.4 Calibration: the method knows when it is unsure

The pooled weight of the chosen answer serves as a confidence. On **test**, never used for any choice:

Confidence	**< 0.5**	**0.5–0.7**	**0.7–0.85**	**0.85–0.95**	**≥ 0.95**
Questions	14	98	100	95	160
Accuracy	28.6%	58.2%	80.0%	95.8%	**98.1%**



Accuracy rises monotonically with confidence: the 160 most confident questions (34%) are answered at 98.1%, the 112 below 0.7 at 54.5%. Validation shows the same curve (27.3% → 97.7%). A deployment that may abstain could drop the least confident third and answer the rest at about 95%.

3.5 Validation as a proxy, and its optimism

Using only test **inputs** (candidate texts, no references), the vote structure of hard questions matches across splits: strict 3-vote majority 41.9 / 42.5 / 42.0%, strict 2-vote majority 32.4 / 32.7 / 32.5%, true tie 25.8 / 24.8 / 25.5% for train / validation / test. The validation hard subset is therefore a legitimate proxy. This check informed no design decision.

Validation was nonetheless reused across roughly fifteen configurations, and with 355 hard questions (±1.8 points standard error) repeated comparison makes its scores optimistic. The frozen method scores 86.20% on the validation hard subset but **83.30% on test**; the 2.9-point gap is within sampling noise but consistent with mild adaptive overfitting. I therefore treat **83.30% as the only unbiased estimate**, and made every decision on train cross-validation.

3.6 More computation did not pay off

All decided on train cross-validation alone:

Attempt	Train CV hard	vs. 82.26
Within-question features	82.02	−0.24
Gradient-boosted second stage (depth 2)	82.40	+0.14
Ensemble of four scorers (best of 11 combinations)	81.93	−0.33
One-hidden-layer MLP, h = 64 / h = 32	80.37 / 80.13	−1.89 / −2.13
Trained on hard questions only	81.69	−0.57
Process-consistency features (§5.4)	83.11	+0.85 (+0.57 vs. re-tuned baseline)

Each is a reasonable idea, and each fails at 4,755 training questions and 1,050 input dimensions. The MLP is the clearest case: more capacity, 966 s to train instead of about one minute, and 1.9 points worse. The 1024-d embedding already supplies non-linear structure, so the extra non-linearity adds variance rather than signal.

---

4. Error analysis

The frozen method makes 49 errors on the 798 validation questions (25 majority-wrong, 18 true-tie, 6 majority-right). **None of the 49 chosen solutions contains a detectable arithmetic error.** In 47 of 49, the chosen solution leaves the same question numbers unused as the correct one (mean unused numbers: 0.081 for correct, 0.131 for wrong candidates), and 30 of 49 chosen solutions are *longer* than the correct one, so neither cue discriminates. Reading the 25 majority-wrong cases by hand:

Failure mode	~n	Examples
Dropped a condition or quantity	10	forgot to multiply by two bodyguards; omitted one child's 13 eggs; ignored the four baking trays; applied 20 episodes to all ten seasons
Misread the question's semantics	13	read "20% of the price" as "20% off"; "1/4 more than" as "add 1/4"; "50% longer" as "50% in total"; simple interest as compounding

In both groups **the arithmetic is correct, the formatting is clean, and the reasoning is internally consistent**: the solutions solve a slightly different problem, exactly the failure mode the assignment guide warns about. This explains the small value of verification. Across the training corpus, equation checking finds arithmetic mistakes in only about 1% of candidates, and among 367 majority-wrong training questions only 27 (7%) have a majority candidate that demonstrably miscalculated.

---

5. Discussion

5.1 The strongest feature is a property of the dataset

Training references are **100% integers**, so "is the answer an integer?" is the most heavily weighted feature. This is a property of GSM8K, not of reasoning. Simply dropping non-integer answers before voting lifts validation hard accuracy from 70.70% to **74.37%** with no learning, so any claim that a verifier "understands reasoning" must be net of this prior.

5.2 Length behaves differently across and within questions

Across questions, length carries no signal once the embedding is present (§3.2), and in a text-only model it is weighted negatively (−0.46). Within a question, among vote-tied candidates, longer solutions are more often right: on 545 training tie questions the longest is correct 50.64% of the time versus 28.26% for the shortest. A plausible reading is that hard questions attract both longer solutions and more errors, while within a question more text often means more conditions were used. Either way, it is a surface correlation the embedding subsumes.

5.3 What makes a wrong answer look convincing

From §4: **a wrong answer is convincing when its arithmetic is flawless and only its interpretation is wrong**. Every feature I built (parseability, integer-ness, numeric coverage, equation checking, answer–process consistency) measures the correctness of *execution*, while the failures are errors of *comprehension*. The feature closest to the line, whether the stated answer equals the last computed step (AUC 0.607, the best verification feature), detects inconsistency rather than misinterpretation.

5.4 A mechanism that is real but too small to exploit

The error analysis suggested a testable mechanism: if a wrong solution stops one step early, its answer should appear as an **intermediate** value inside a solution that carries on. Verified on train hard questions (base rate 50.7%): candidates whose answer appears as another solution's intermediate step are correct only **28.4%** of the time (n = 1,231), while those that carry past a competitor's answer are correct **66.9%** (n = 1,491). Single features from this block reach AUC 0.661, and the block alone scores 73.52% on the validation hard subset, above voting.

Yet adding it to the full model moved train CV from 82.54% to 83.11%, **+0.57 points** on a like-for-like comparison with `C` and `T` tuned on the deployed metric for both (+0.85 against the frozen configuration). Before running the experiment I wrote down an adoption threshold of +1.0 point, so I did not adopt it. The mechanism is real, but at this data scale its payoff is indistinguishable from noise. It is the most informative negative result here, because it shows how little headroom remains.

5.5 A methodology flaw I did not fix on purpose

`C` was selected on the **raw argmax** CV metric, while the deployed decision includes answer pooling. Selecting on the deployed metric would give `C = 0.1, T = 1.5` and move train CV from 82.26 to 82.54 (+0.28). I did not adopt it. The adoption threshold of §5.4 was fixed before the test result was known, and re-selecting hyper-parameters after seeing test (83.30%, below the validation estimate) and then re-running the test bank would amount to selection on test data, even if the re-selection used only train. A 0.28-point difference is well inside noise anyway (§3.5).

5.6 Choosing the simpler model when scores are close

Where configurations differed by less than 0.1 points on train CV, I preferred the simpler one: the frozen model uses three evidence blocks, omitting the within-question block that contributes nothing. It also calibrates better than the four-block finalist, which puts 226 of 355 hard questions above 0.95 confidence while answering only 94.2% of them correctly; the frozen model's confidence curve is monotone (§3.4). This is a conventional rule, not a project requirement.

---

6. Resources and reproducibility

Item	Limit	Measured
Inference time (prediction + scoring, 4 threads)	≤ 30 s	**2.02 s** (median of three, validation's 798 questions, both processes started fresh)
Inference assets	≤ 16 MiB	**0.0134 MiB** (13,594 B weights + 487 B config)
Inference dependencies	—	NumPy and the standard library only (import chain inspected)
Training	CPU or ≤ RTX 4090	CPU only; 65 s per model including the 6 × 5-fold search over `C`
Local hardware	report	AMD Ryzen 9 7945HX (32 threads), 15.7 GB RAM, Windows 11

`model.npz` sha256: `b5ddef691e285872f21983d2c52c047adb1f2f4543b55be12d5ec54283319818`. Re-running the two training commands from scratch reproduced the checkpoint **byte-for-byte**, since the only search (`C` over five folds with a fixed seed) and the choice of `T` are deterministic. A synthetic audit bank of 137 questions (shuffled order, new IDs, an unrelated question count) runs through `predict.py --split audit` and yields 137 valid indices, so the method depends on neither IDs, order nor sample count.

Data use. Training scripts read only the train split; the ablation additionally reads validation. Test references were opened only after the method was frozen, for final scoring and the post-hoc breakdowns of §3.3–3.4.

AI disclosure. This project was carried out with substantial help from Claude (Anthropic), used as a coding and analysis assistant throughout. Claude:

- wrote the code: feature extraction, the numeric verifier, training, evaluation, ablation and figure scripts;
- proposed most method components and experiments, including the listwise loss, answer-level pooling, the verification and process-consistency features, and the probes that produced the negative results;
- ran the experiments and summarised the results after each step;
- carried out the failure analysis, quantifying the error types and reading and grouping the majority-wrong cases;
- drafted this report and its figures, and cross-checked the reported numbers against the experiment records.

I directed the work step by step, made the decisions at each choice point (for example the tie-breaking rule, the final configuration, and stopping further tuning instead of re-running the test bank), and reviewed the results and the report. I take responsibility for all of its content.

---

7. Conclusion

A linear scorer over supplied embeddings, text statistics and arithmetic checks, plus one structural decision (pooling candidate scores by parsed answer), lifts hard-question accuracy from **70.70%** (majority voting) to **86.20%** on validation and **83.30%** on the official test bank. It needs a 13 KB checkpoint and two seconds of CPU, and its confidence signal is monotone on unseen data. The embedding contributes most of the signal, the verifier a little, and length and formatting nothing; richer models and features all failed to improve on train cross-validation. The remaining errors are solutions that reason correctly about the wrong reading of the question. That is a boundary verification without comprehension could not cross, and measuring where it lies is, I think, the most useful result here.